

Projet MLOps CI/CD avec Docker, GitLab et Harbor

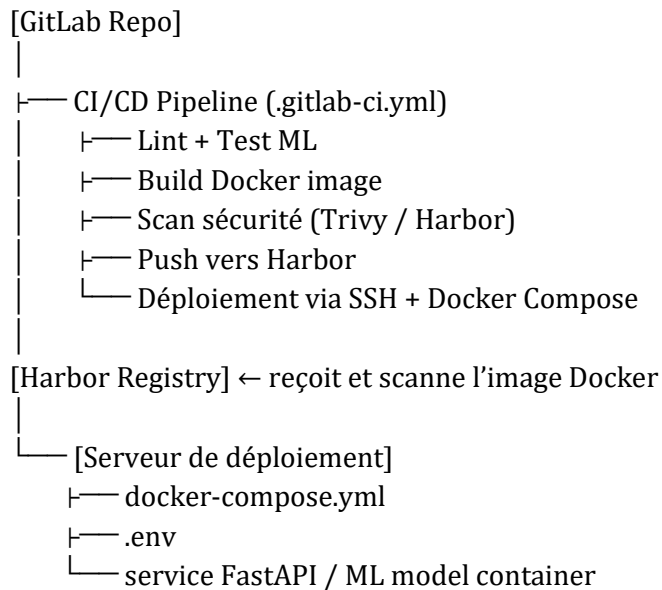
Sujet

MLOps CI/CD : pipeline automatisé de Machine Learning avec Docker, GitLab CI/CD et Harbor — déploiement via Docker Compose

Objectif global

Mettre en place une chaîne CI/CD complète pour un projet de Machine Learning utilisant GitLab, Harbor et Docker Compose. Le pipeline doit automatiser le cycle complet : Code ML → Build & tests → Image Docker → Scan sécurité → Push sur Harbor → Déploiement Compose automatique sur un serveur distant (staging / prod).

Architecture cible



Stack technique

- GitLab CE ou GitLab.com (CI/CD)
- Harbor (registre privé + scan sécurité)
- Docker & Docker Compose
- Python 3.11 + FastAPI
- Framework ML : scikit-learn ou PyTorch
- Trivy (scan vulnérabilités)
- pytest, flake8, black
- Grafana + Prometheus (optionnel : monitoring)

Plan sur 3 mois (12 semaines)

Semaines 1–2 : Setup et spécification

Choix du cas d'usage ML, dataset, métriques, initialisation projet + repo GitLab.

Semaines 3–4 : Code ML & containerisation

Scripts de preprocessing, entraînement, export du modèle, serveur FastAPI et Dockerfile.

Semaines 5–6 : Pipeline GitLab CI/CD

Lint, test, build Docker, push Harbor, scan sécurité et déploiement Compose.

Semaines 7–8 : Déploiement via Docker Compose

Création du docker-compose.yml et déploiement auto via SSH.

Semaines 9–10 : Sécurité et monitoring

Scan Harbor/Trivy, dashboard Grafana, secrets CI.

Semaines 11–12 : Documentation & rapport

Rapport technique complet + démo vidéo.

Structure recommandée du projet

```
/.gitlab-ci.yml
/README.md
/docker-compose.yml
/.env.example
/src/
├── train.py
├── predict.py
├── app.py
/tests/
├── test_train.py
├── test_api.py
/docker/
├── Dockerfile
/docs/
├── cahier_des_charges.md
├── architecture.md
├── rapport_final.md
/monitoring/
├── prometheus.yml
├── grafana/
```

Énoncé par M. Bonitah RAMBELOSON
Consultant DevOps | Cloud Engineer | MLOps Practitioner

Cas d'usage ML suggérés

- Classification de texte (détection de spam)
- Prédiction de churn client
- Détection d'anomalies de séries temporelles
- Classification d'images (MNIST, CIFAR-10)
- Recommandation de produits

Évaluation finale (proposition)

Critère	Pondération
Fonctionnalité complète du pipeline CI/CD	40%
Gestion du registre Harbor (sécurité, push, scan)	20%
Qualité du code ML + tests	15%
Déploiement automatique Docker Compose	15%
Documentation & démonstration	10%